JavaScript Master Handbook

How to use this handbook

This handbook is built to help you **memorize**, **understand**, and **practice** JavaScript.

For each topic, follow this pattern: 1. Read the idea. 2. Memorize the short rule. 3. Study the example. 4. Type the code yourself. 5. Change the example and test it.

---

1. What JavaScript is

JavaScript is a programming language mainly used to make web pages interactive.

You can use JavaScript in: - the browser - servers with Node.js - mobile and desktop apps

Easy memory idea

HTML = structure
CSS = style
JavaScript = behavior

Example:

```
console.log("Hello JavaScript");
```

---

2. How to write JavaScript

Output

```
console.log("Hello");
alert("Welcome");
```

Comments

```
// This is a single-line comment

/*
This is a multi-line comment
*/
```

Memorize

Comments are notes for humans, not instructions for the computer.

---

3. Variables

Variables store data.

Three ways

```
var oldWay = "old";
let age = 25;
const country = "Sri Lanka";
```

Difference

- `var` → old style, function-scoped
- `let` → can change value
- `const` → cannot be reassigned

Memorize rule

- Use `const` by default
- Use `let` when value changes
- Avoid `var`

Example:

```
let score = 10;
score = 20;

const pi = 3.14;
// pi = 4; // Error
```

---

4. Data types

Primitive types

- string
- number
- boolean
- undefined
- null
- bigint
- symbol

Reference type

- object

Example:

```
let name = "Sameera";       // string
let marks = 85;             // number
let isPassed = true;        // boolean
let x;                      // undefined
let y = null;               // null
```

Check type:

```
console.log(typeof name);
console.log(typeof marks);
console.log(typeof isPassed);
```

Memorize

`typeof` tells the type.

---

5. Strings

Strings are text values.

```
let firstName = "John";
let lastName = 'Doe';
let message = `Hello ${firstName} ${lastName}`;
```

Useful string methods

```
let text = "JavaScript";

console.log(text.length);
console.log(text.toUpperCase());
console.log(text.toLowerCase());
console.log(text.includes("Script"));
console.log(text.slice(0, 4));
console.log(text.replace("Java", "Type"));
```

Memorize

- `length` → number of characters
- `includes()` → checks if text exists
- `slice()` → cuts part of string
- template literals use backticks `

Practical example:

```
let user = "sameera";
console.log(`Welcome, ${user.toUpperCase()}!`);
```

---

6. Numbers

```
let a = 10;
let b = 3;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
console.log(a ** b);
```

Important methods

```
let num = 12.6789;
console.log(num.toFixed(2));
console.log(Math.round(num));
console.log(Math.floor(num));
console.log(Math.ceil(num));
console.log(Math.max(5, 9, 2));
console.log(Math.min(5, 9, 2));
console.log(Math.random());
```

Memorize

- `%` gives remainder
- `toFixed(2)` gives 2 decimal places
- `Math.random()` gives random number from 0 to less than 1

Practical example:

```
let price = 199.456;
console.log(`Price: $${price.toFixed(2)}`);
```

---

7. Operators

Arithmetic

```
+ - * / % **
```

Assignment

```
= += -= *= /=
```

Comparison

```
== === != !== > < >= <=
```

Logical

```
&& || !
```

Important difference

```
console.log(5 == "5");   // true
console.log(5 === "5");  // false
```

Memorize

- == compares value only
- === compares value and type
- Prefer ===

---

8. Type coercion

JavaScript sometimes automatically converts one type into another.

```
console.log("5" + 2); // "52"
console.log("5" - 2); // 3
console.log(Number("123"));
console.log(String(456));
console.log(Boolean(1));
```

Memorize

- + with string often joins text
- Use explicit conversion when possible

---

9. Conditionals

if

```
let age = 20;

if (age >= 18) {
```

```
  console.log("Adult");
}
```

if else

```
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

else if

```
let marks = 75;

if (marks >= 90) {
  console.log("A");
} else if (marks >= 75) {
  console.log("B");
} else {
  console.log("C");
}
```

ternary operator

```
let result = age >= 18 ? "Adult" : "Minor";
console.log(result);
```

Memorize

Ternary is short form of if-else.

---

10. switch

Use `switch` for many exact matches.

```
let day = 2;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  default:
    console.log("Unknown day");
}
```

Memorize

`break` stops the next cases from running.

---

11. Loops

for loop

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

while loop

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

do while

```
let x = 1;
do {
  console.log(x);
  x++;
} while (x <= 3);
```

for...of

```
let fruits = ["apple", "banana", "mango"];
for (const fruit of fruits) {
  console.log(fruit);
}
```

for...in

```
let user = { name: "Sam", age: 25 };
for (const key in user) {
  console.log(key, user[key]);
}
```

Memorize

- `for` → known number of repetitions
- `while` → repeat while condition is true
- `for...of` → values in iterable
- `for...in` → keys in object

12. Functions

Functions are reusable blocks of code.

Function declaration

```
function greet(name) {
  return `Hello ${name}`;
}

console.log(greet("Sameera"));
```

Function expression

```
const add = function(a, b) {
  return a + b;
};
```

Arrow function

```
const multiply = (a, b) => a * b;
console.log(multiply(3, 4));
```

Default parameters

```
function welcome(name = "Guest") {
  return `Welcome ${name}`;
}
```

Memorize

- function takes input = parameters
- function gives output = return
- arrow functions are shorter

Practical example:

```
function calculateTotal(price, qty) {
  return price * qty;
}

console.log(calculateTotal(120, 3));
```

13. Scope

Scope means where a variable can be used.

Global scope

```
let appName = "My App";
```

Local scope

```
function test() {
  let message = "inside";
  console.log(message);
}
```

Block scope

```
if (true) {
  let a = 10;
  const b = 20;
}
```

Memorize

- `let` and `const` are block-scoped
- `var` is function-scoped

---

14. Arrays

Arrays store multiple values in order.

```
let fruits = ["apple", "banana", "mango"];
console.log(fruits[0]);
console.log(fruits.length);
```

Common methods

```
fruits.push("orange");
fruits.pop();
fruits.unshift("grape");
fruits.shift();
console.log(fruits.includes("banana"));
console.log(fruits.indexOf("mango"));
```

Loop through array

```
for (const fruit of fruits) {
  console.log(fruit);
}
```

Memorize

- `push()` add to end
- `pop()` remove from end
- `unshift()` add to beginning
- `shift()` remove from beginning

Practical example:

```
let cart = ["Pen", "Book", "Pencil"];
cart.push("Bag");
console.log(cart);
```

---

15. Array higher-order methods

These are very important for modern JavaScript.

forEach

```
[1, 2, 3].forEach(num => console.log(num));
```

map

```
let nums = [1, 2, 3];
let doubled = nums.map(n => n * 2);
console.log(doubled);
```

filter

```
let ages = [12, 18, 25, 15];
let adults = ages.filter(age => age >= 18);
console.log(adults);
```

find

```
let users = [{id: 1}, {id: 2}];
let user = users.find(u => u.id === 2);
console.log(user);
```

reduce

```
let prices = [100, 200, 300];
let total = prices.reduce((sum, price) => sum + price, 0);
console.log(total);
```

some and every

```
console.log([1, 2, 3].some(n => n > 2));
console.log([1, 2, 3].every(n => n > 0));
```

Memorize shortcut

- `map` → transform
- `filter` → keep matching items
- `find` → first matching item
- `reduce` → combine into one value
- `forEach` → just loop

Practical example:

```
const items = [
  { name: "Pen", price: 10 },
  { name: "Book", price: 50 }
];

const total = items.reduce((sum, item) => sum + item.price, 0);
console.log(total);
```

---

16. Objects

Objects store data as key-value pairs.

```
let user = {
  name: "Sameera",
  age: 25,
  isStudent: true
};
```

Access object data

```
console.log(user.name);
console.log(user["age"]);
```

Add or update

```
user.city = "Colombo";
user.age = 26;
```

Delete

```
delete user.isStudent;
```

Memorize

- dot notation: `user.name`
- bracket notation: `user["name"]`

---

17. Object methods and this

Objects can have functions inside them.

```
const user = {
  firstName: "John",
  lastName: "Doe",
  getFullName: function() {
    return `${this.firstName} ${this.lastName}`;
  }
};

console.log(user.getFullName());
```

Memorize

`this` usually refers to the object calling the method.

---

18. Destructuring

Destructuring extracts values easily.

Array destructuring

```
const colors = ["red", "green", "blue"];
const [first, second] = colors;
console.log(first, second);
```

Object destructuring

```
const user = { name: "Sam", age: 22 };
const { name, age } = user;
console.log(name, age);
```

Memorize

Destructuring = unpack values quickly.

---

19. Spread and rest operators

Spread

```
const a = [1, 2];
const b = [...a, 3, 4];
console.log(b);
```

Rest

```
function sum(...nums) {
  return nums.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3, 4));
```

Memorize

- spread `...` expands
- rest `...` collects

---

20. Template literals

```
let name = "Sameera";
let age = 25;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Memorize

Use backticks for variable insertion and multiline strings.

---

21. Truthy and falsy values

Falsy values

- `false`
- `0`
- `""`
- `null`
- `undefined`
- `NaN`

Everything else is usually truthy.

Example:

```
if ("") {
  console.log("runs");
} else {
  console.log("empty string is falsy");
}
```

---

22. Optional chaining and nullish coalescing

Optional chaining

```
const user = { profile: { name: "Sam" } };
console.log(user.profile?.name);
console.log(user.address?.city);
```

Nullish coalescing

```
let username = null;
console.log(username ?? "Guest");
```

Memorize

- `?.` prevents errors when a property may not exist
- `??` uses default only for `null` or `undefined`

---

23. Date and time

```
const now = new Date();
console.log(now);
console.log(now.getFullYear());
console.log(now.getMonth());
console.log(now.getDate());
```

Note

Month starts from `0`, so January is `0`.

Practical example:

```
const today = new Date();
console.log(`${today.getDate()}/${today.getMonth() +
1}/${today.getFullYear()}`);
```

---

24. Error handling

Use `try...catch` to handle errors.

```
try {
  let result = 10 / 0;
  console.log(result);
```

```
} catch (error) {
  console.log("Something went wrong");
}
```

Custom error example:

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}
```

Memorize

- `try` → risky code
- `catch` → handle error
- `throw` → create error

---

25. DOM basics

DOM means Document Object Model. It lets JavaScript control HTML.

Select elements

```
const title = document.getElementById("title");
const button = document.querySelector("button");
const items = document.querySelectorAll("li");
```

Change content

```
title.textContent = "New Title";
```

Change HTML

```
title.innerHTML = "<span>Hello</span>";
```

Change style

```
title.style.color = "red";
```

Memorize

- `getElementById()` → select by id
- `querySelector()` → first matching element
- `querySelectorAll()` → all matching elements

---

26. Events

Events are user actions like click, input, submit.

```
const btn = document.querySelector("button");

btn.addEventListener("click", function() {
  console.log("Button clicked");
});
```

Input example:

```
const input = document.querySelector("input");

input.addEventListener("input", function(e) {
  console.log(e.target.value);
});
```

Memorize

`addEventListener()` listens for actions.

---

27. Forms

```
const form = document.querySelector("form");

form.addEventListener("submit", function(e) {
  e.preventDefault();
  console.log("Form submitted without page reload");
});
```

Memorize

`e.preventDefault()` stops default browser behavior.

---

28. Classes and classList

```
const box = document.querySelector(".box");
box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("dark");
```

Memorize

Use `classList` to manage CSS classes.

---

29. setTimeout and setInterval

```
setTimeout(() => {
  console.log("Runs once after 2 seconds");
}, 2000);

const id = setInterval(() => {
  console.log("Runs repeatedly");
}, 1000);

clearInterval(id);
```

Memorize

- `setTimeout` → once later
- `setInterval` → repeat again and again

---

30. JSON

JSON is a text format for data exchange.

```
const user = { name: "Sam", age: 25 };
const jsonText = JSON.stringify(user);
console.log(jsonText);

const backToObject = JSON.parse(jsonText);
console.log(backToObject.name);
```

Memorize

- `JSON.stringify()` object to JSON string
- `JSON.parse()` JSON string to object

---

31. Local storage

Store small data in the browser.

```
localStorage.setItem("username", "Sameera");
console.log(localStorage.getItem("username"));
localStorage.removeItem("username");
```

Object example:

```
const settings = { theme: "dark" };
localStorage.setItem("settings", JSON.stringify(settings));
const saved = JSON.parse(localStorage.getItem("settings"));
console.log(saved.theme);
```

---

32. Asynchronous JavaScript

JavaScript can do tasks that finish later.

Callback

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

greet("Sam", function() {
  console.log("Done");
});
```

Promise

```
const promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("Completed");
  } else {
    reject("Failed");
  }
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

async/await

```
function getData() {
  return new Promise(resolve => {
    setTimeout(() => resolve("Data loaded"), 1000);
  });
}

async function showData() {
  const result = await getData();
  console.log(result);
}
```

```
showData();
```

Memorize

- callback = function passed to another function
- promise = future result
- `async/await` = cleaner way to handle promises

---

33. Fetch API

Use fetch to get data from an API.

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

With async/await:

```
async function getPost() {
  try {
    const response = await
fetch("https://jsonplaceholder.typicode.com/posts/1");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```

---

34. ES6+ features you must master

- `let` and `const`
- arrow functions
- template literals
- destructuring
- spread/rest
- default parameters
- modules
- promises
- async/await

- optional chaining
- nullish coalescing

---

35. Modules

Split code into files.

Export

```
export const name = "Sam";
export function add(a, b) {
  return a + b;
}
```

Import

```
import { name, add } from "./app.js";
console.log(name);
console.log(add(2, 3));
```

---

36. Basic OOP in JavaScript

Class

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    return `Hi, I am ${this.name}`;
  }
}

const p1 = new Person("Sameera", 25);
console.log(p1.greet());
```

Inheritance

```
class Student extends Person {
  constructor(name, age, course) {
    super(name, age);
    this.course = course;
```

```
  }
}
```

Memorize

- class = blueprint
- object = instance of class
- `constructor` runs when object is created
- `extends` means inheritance

---

37. Important built-in methods

String methods

- `includes()`
- `slice()`
- `trim()`
- `replace()`
- `split()`

Array methods

- `push()`
- `pop()`
- `map()`
- `filter()`
- `find()`
- `reduce()`
- `join()`

Object methods

```
const user = { name: "Sam", age: 25 };
console.log(Object.keys(user));
console.log(Object.values(user));
console.log(Object.entries(user));
```

---

38. Practical mini projects

1. Even or odd checker

```
function isEven(num) {
  return num % 2 === 0;
}

console.log(isEven(4));
```

2. Vowel counter

```
function countVowels(str) {
  return (str.match(/[aeiou]/gi) || []).length;
}

console.log(countVowels("Hello World"));
```

3. Shopping cart total

```
function summarizeCart(items) {
  const total = items.reduce((sum, item) => sum + item.price * item.qty, 0);
  const count = items.reduce((sum, item) => sum + item.qty, 0);
  return `Total: $${total.toFixed(2)} (${count} items)`;
}
```

4. Simple to-do list idea

- take text from input
- create `li`
- append to list
- remove item on click

5. Digital clock idea

- use `new Date()`
- show hours, minutes, seconds
- update every second with `setInterval`

---

39. Common beginner mistakes

- forgetting `return`
- using = instead of ===
- mixing up == and ===
- not calling a function with `()`

- misspelling method names
- forgetting array indexes start at `0`
- forgetting `break` in `switch`
- writing template literal with quotes instead of backticks
- using `getLastRow` instead of `getLastRow()` style mistakes in programming generally

---

40. Best practices

- use meaningful variable names
- keep functions small
- use `const` by default
- prefer `===`
- format code neatly
- test small parts one by one
- read error messages carefully
- practice by building small projects

---

41. Easy memory map

JavaScript core roadmap

1. Variables
2. Data types
3. Operators
4. Conditionals
5. Loops
6. Functions
7. Arrays
8. Objects
9. DOM
10. Events
11. Async JavaScript
12. APIs
13. Projects

One-line memory formula

Store → Check → Repeat → Reuse → Organize → Interact → Fetch → Build

42. 30-day JavaScript mastery plan

Week 1: Foundations

- Day 1: variables and data types
- Day 2: strings and numbers
- Day 3: operators and type coercion
- Day 4: if, else, switch
- Day 5: loops
- Day 6: functions
- Day 7: revision + 10 small problems

Week 2: Arrays and objects

- Day 8: arrays basics
- Day 9: array methods
- Day 10: objects
- Day 11: object methods and `this`
- Day 12: destructuring, spread, rest
- Day 13: practice problems
- Day 14: mini project

Week 3: DOM and browser JavaScript

- Day 15: selecting elements
- Day 16: changing text, HTML, style
- Day 17: events
- Day 18: forms
- Day 19: classList, localStorage
- Day 20: date and timers
- Day 21: mini project

Week 4: Advanced basics

- Day 22: callbacks and promises
- Day 23: async/await
- Day 24: fetch API
- Day 25: JSON
- Day 26: modules
- Day 27: classes and OOP

- Day 28: revision
- Day 29: build one full mini project
- Day 30: solve 20 problems without notes

---

43. How to truly master JavaScript

To master JavaScript, do these every day:

1. Write code by hand.
2. Predict output before running.
3. Fix errors yourself first.
4. Build mini projects.
5. Revise old topics often.
6. Explain concepts in your own words.

Golden rule

Do not only read JavaScript. Type JavaScript. Break JavaScript. Fix JavaScript.

---

44. Practice questions

Basic

1. Create two variables and print their sum.
2. Check whether a number is positive or negative.
3. Print numbers from 1 to 10.
4. Write a function to square a number.
5. Count vowels in a string.

Intermediate

6. Reverse a string.
7. Find the largest number in an array.
8. Filter only even numbers.
9. Find total cart value from array of objects.
10. Create an object for a student and print values.

DOM

11. Change heading text when button is clicked.
12. Read input value and show it on page.

13. Add list items dynamically.
14. Save theme in localStorage.
15. Build a calculator UI.

Async

16. Fetch user data from an API.
17. Show loading text before data arrives.
18. Handle errors from fetch.

---

45. Final revision sheet

Most important things to memorize first

- `let`, `const`
- strings, numbers, booleans
- `if`, `else`, `switch`
- `for`, `while`
- functions and return
- arrays and objects
- `map`, `filter`, `reduce`
- DOM selection
- events
- async/await
- fetch

Most important methods

- `toUpperCase()`
- `includes()`
- `slice()`
- `push()`
- `pop()`
- `map()`
- `filter()`
- `find()`
- `reduce()`
- `JSON.stringify()`
- `JSON.parse()`

---

46. Final advice

You do not need to memorize everything at once.

Memorize in layers: - first understand the idea - then memorize the pattern - then practice small examples - then build projects

The students who master JavaScript are usually not the ones who read the most. They are the ones who **practice consistently**.